```
        if result != null then
          return
result                                                    #8
  return
null
      #9
```

#1 Search returns the tree leaf that contains a target point, if the point is stored in the tree – returns `null` otherwise. We explicitly pass the root of the (sub)tree we want to search, so we can reuse this function for sub-trees.
#2 Checks if `node` is a leaf or an internal node.
#3 If `node` is a leaf, goes through all the points held, and checks if any matches `target`.
#4 If a match is found, returns current leaf.
#5 Otherwise, if we are traversing an internal node, goes through all its children, and checks which ones could contain target: in other words, for each children `childNode` we check the distance between its centroid and the target point, and if this is smaller than the bounding envelope's radius of `childNode`, we recursively traverse the `childNode`.
#6 Check if `childNode` could contain `target`, i.e. if `target` is within `childNode`'s bounding envelope. See listing 10.4 for an implementation.
#7/#8 if that's the case, perform a recursive search on `childNode`'s branch, and if the result is an actual node (and not `null`), we have found what we were looking for and we can return.
#9 If no child of current node could contain `target`, or if we are at a leaf and no point matches `target`, then we end